

```
create database Mydatabase
```

```
use mydatabase
```

```
-- Creating the Authors table
```

```
CREATE TABLE Authors (  
    AuthorID INT PRIMARY KEY IDENTITY(1,1),  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    BirthYear INT  
);
```

```
-- Inserting sample data into Authors table
```

```
INSERT INTO Authors (FirstName, LastName, BirthYear) VALUES  
( 'George', 'Orwell', 1903),  
( 'J.K.', 'Rowling', 1965),  
( 'J.R.R.', 'Tolkien', 1892),  
( 'Agatha', 'Christie', 1890),  
( 'Stephen', 'King', 1947);
```

```
-- Creating the Categories table
```

```
CREATE TABLE Categories (  
    CategoryID INT PRIMARY KEY IDENTITY(1,1),  
    CategoryName NVARCHAR(100) NOT NULL  
);
```

```
-- Inserting sample data into Categories table
```

```
INSERT INTO Categories (CategoryName) VALUES  
( 'Science Fiction'),
```

('Fantasy'),  
('Mystery'),  
('Horror'),  
('Non-Fiction');

-- Creating the Books table

```
CREATE TABLE Books (  
    BookID INT PRIMARY KEY IDENTITY(1,1),  
    Title NVARCHAR(255) NOT NULL,  
    AuthorID INT,  
    CategoryID INT,  
    PublicationYear INT,  
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID) ON DELETE CASCADE,  
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) ON DELETE  
    CASCADE  
);
```

--inserting data into books table

```
INSERT INTO Books (Title, AuthorID, CategoryID, PublicationYear) VALUES  
('1984', 1, 1, 1949),  
('Harry Potter and the Sorcerer's Stone', 2, 2, 1997),  
('The Hobbit', 3, 2, 1937),  
('Murder on the Orient Express', 4, 3, 1934),  
('The Shining', 5, 4, 1977);
```

-- Creating the Members table

```
CREATE TABLE Members (  
    MemberID INT PRIMARY KEY IDENTITY(1,1),  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,
```

```
MembershipDate DATE NOT NULL  
);
```

```
-- Inserting sample data into members table
```

```
INSERT INTO Members (FirstName, LastName, MembershipDate) VALUES  
( 'Alice', 'Smith', '2023-01-15'),  
( 'Bob', 'Johnson', '2022-05-22'),  
( 'Charlie', 'Brown', '2021-09-10'),  
( 'David', 'Wilson', '2020-11-30'),  
( 'Emma', 'Davis', '2019-07-18');
```

```
-- Creating the Borrowing table
```

```
CREATE TABLE Borrowing (  
    BorrowingID INT PRIMARY KEY IDENTITY(1,1),  
    MemberID INT,  
    BookID INT,  
    BorrowDate DATE NOT NULL,  
    ReturnDate DATE,  
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID) ON DELETE  
    CASCADE,  
    FOREIGN KEY (BookID) REFERENCES Books(BookID) ON DELETE CASCADE  
);
```

```
-- Inserting sample data into borrows table
```

```
INSERT INTO Borrowing (MemberID, BookID, BorrowDate, ReturnDate) VALUES  
(1, 1, '2024-02-01', '2024-02-15'),  
(2, 2, '2024-01-10', NULL),  
(3, 3, '2024-01-25', '2024-02-10'),  
(4, 4, '2024-02-05', NULL),  
(5, 5, '2024-02-15', '2024-02-25');
```

-----PART 2 -----

```
select BusinessEntityId, JobTitle, HireDate
from humanresources.employee
```

--2

```
select ProductNumber, Name, SellStartDate
from production.product
```

--Filtering

--1

```
select * from humanresources.employee
where vacationhours > 50
```

--2

```
select * from production.product
where standardcost < 100
```

--Aggreagate

--1

```
select count(*) as 'Total count of products' from production.product
```

--2

```
select min(listprice) as 'Min ListPrice' from production.product
```

--Grouping by

--1

```
SELECT ProductModelID, COUNT(*) AS ProductCount
```

```
FROM Production.Product  
GROUP BY ProductModelID
```

```
--2
```

```
select sum (TotalDue) as 'TotalDue', SalesPersonID  
from sales.salesorderheader  
group by salesPersonId
```

```
--OrderBy
```

```
--1
```

```
SELECT Name, Weight  
FROM Production.Product  
ORDER BY Weight ASC
```

```
--2
```

```
SELECT LastName, FirstName  
FROM Person.Person  
ORDER BY LastName
```

```
--Having
```

```
--1 finding the count of product then extracting the ones that are greater than 10
```

```
SELECT ProductModelID as 'ModelID', COUNT(*) AS ProductCount  
FROM Production.Product  
GROUP BY ProductModelID  
HAVING COUNT(*) > 10  
ORDER BY ProductModelID
```

--2 using the sum function that will compare the sum to the given value(1000)

```
SELECT SalesPersonID, SUM(TotalDue) AS TotalSalesAmount
FROM Sales.SalesOrderHeader
GROUP BY SalesPersonID
HAVING SUM(TotalDue) > 50000
ORDER BY TotalSalesAmount DESC
```

--Top

--1

```
SELECT TOP 10 Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC
```

--2

```
SELECT TOP 5 CustomerID, SUM(TotalDue) AS TotalSalesAmount
FROM Sales.SalesOrderHeader
GROUP BY CustomerID
ORDER BY TotalSalesAmount DESC
```

--Joins: joining multiple tables through their common columns

--1

```
SELECT e.FirstName, e.LastName, d.Name 'DepartmentName'
FROM person.person AS e
JOIN HumanResources.EmployeeDepartmentHistory AS edh
    ON e.BusinessEntityID = edh.BusinessEntityID
JOIN HumanResources.Department AS d
    ON edh.DepartmentID = d.DepartmentID
```

ORDER BY e.LastName, e.FirstName

--2

```
select p.Name, sc.Name as 'SubCategory'
from production.product as p
join production.productsubcategory as sc
on p.productsubcategoryid = sc.productsubcategoryid
```

--Like

--1

```
select * from production.product
where name like '%_road_%'
```

--2 pattern that ends with cycles therefore we will have \_ for the starting

```
select * from purchasing.vendor
where name like '_%cycles'
```

--Between (range)

--1

```
select * from humanresources.employee
where hiredate between '2017-01-01' and '2017-12-31'
```

--2

```
select * from production.product
where listprice between '200' and '800'
```

--AND/OR

--1 both condition must be met

```
select * from humanresources.employee  
where gender ='F' and VacationHours > 30
```

--2 when either condition is met, then it will displayes

```
select * from production.product  
where color= 'Black' or safetyStockLevel < 1000
```

--isnull/null

--1

```
select * from person.person  
where suffix is null
```

--2 using the isnull

```
select * from production.product  
where size is null
```

-- Bonus Answers

--case statement along with the join because all columns are not met in  
humanresources.employee

```
select p.firstname, p.lastname,  
case  
    when vacationhours < 20 then 'Low'  
    when vacationhours between 20 and 30 then 'Medium'  
    else 'High'
```



end as VacationCategory

from humanresources.employee as h  
join person.person as p  
on h.businessentityid = p.businessentityid

--String Functions

--1 extract three characters from the left

SELECT Name AS FullName,  
LEFT(Name, 3) AS FirstThreeChars  
FROM Production.Product

--2 using the concat function to join the names with white space in between

SELECT FirstName, LastName,  
CONCAT(FirstName, ' ', LastName) AS FullName  
FROM Person.Person

--Date Functions

--1 using datediff, we are dynamically subtracting the birthdate from the current date and extracting the year only

Select BusinessEntityID, birthdate, datediff(year, birthdate, getdate()) as 'Age' from  
humanresources.employee

--2

SELECT \*  
FROM Sales.SalesOrderHeader

WHERE MONTH(OrderDate) = MONTH(GETDATE())

--MONTH(OrderDate) = MONTH(GETDATE()): Ensures the order was placed in the same month as the current date

--while month(getdate()) compaes it to the current month

## Entity Relation Diagram Of Part One





